

DINÁMICA: ENTREVISTA PARA DESARROLLADOR ANGULAR

Tema: Services, HttpClient, JSON Server y herramientas para probar servicios - 4 personas

Persona	Papel en la dinámica	Cómo actuar
Persona 1	Entrevistadora	Natural y segura. Mantiene el ritmo y conecta las preguntas.
Persona 2	Postulante Angular	Tranquilo y claro. Responde con ejemplos sencillos.
Persona 3	Entrevistador técnico	Curioso. Hace preguntas para aterrizar los ejemplos.
Persona 4	Compañera técnica / QA	Interviene con energía cuando aparece la parte de pruebas.

GUION

Persona 1

“Bien, empezamos con algo básico. Imagina que entras a un proyecto Angular que tiene una tienda y necesita mostrar productos. ¿Qué papel tendría un Service?”

Persona 2

“Yo usaría un Service para sacar esa lógica del componente. Por ejemplo, en vez de hacer que el componente se encargue de todo, el Service se ocupa de obtener los productos y el componente solo los muestra.”

Persona 3

“O sea, ¿el Service sirve como una especie de intermediario entre la interfaz y los datos?”

Persona 2

“Exacto. Así queda más ordenado y, además, el mismo Service se puede reutilizar desde varios componentes. Por eso en Angular es común tener algo como un ProductService para manejar todo lo relacionado con productos.”

Persona 1

Indicacion: Dilo con un tono ligeramente retador, como si estuviera comprobando la respuesta.

“Perfecto. Pero ahora viene la parte que normalmente da un poco más de miedo: necesitamos conectarnos a una API. ¿Cómo entra HttpClient?”

Persona 2

“Ahí entra HttpClient. Con él podemos hacer peticiones HTTP, por ejemplo un GET para pedir los productos, un POST para registrar uno nuevo, un PUT para actualizarlo y un DELETE para eliminarlo.”

Persona 3

“Entonces, si tu Service tuviera que traer los productos, ¿qué pasaría en la práctica?”

Persona 2

“El Service haría la petición. Por ejemplo, tendría un método como `getProducts()` que usa HttpClient para consultar una ruta de productos. Luego el componente llama a ese método y recibe la información para mostrarla en pantalla.”

Persona 4

Indicacion: Dilo con un poco de humor, sin exagerar.

“Ya, pero supongamos que todavía no tenemos un backend real. ¿Nos quedamos mirando la pantalla esperando a que alguien lo termine?”

Persona 2

“No necesariamente. Podemos usar JSON Server. Le damos un archivo `db.json` con nuestros datos y JSON Server los expone como si fueran una API. Así Angular puede consumirla y trabajar con datos reales para la práctica.”

Persona 1

“Dame un ejemplo para que se entienda rápido.”

Persona 2

“Por ejemplo, en db.json podemos tener productos como arroz, aceite y leche. JSON Server puede exponerlos en una ruta como /productos. Entonces Angular hace un GET a /productos y recibe esos productos para mostrarlos en la tienda.”

Persona 3

“Entonces JSON Server simula el backend, pero desde el punto de vista de Angular estamos consumiendo una API. ¿Correcto?”

Persona 2

“Sí. Y eso es justamente lo útil: podemos practicar el flujo completo sin depender todavía de un backend definitivo. Angular hace su parte y JSON Server nos responde con los datos.”

Persona 4

Indicacion: Dilo un poco más animada al entrar a la parte de pruebas.

“Bien, ahora yo quiero saber algo importante: ¿cómo comprobamos que esas peticiones realmente funcionan y no solo “parecen” funcionar?”

Persona 2

“Podemos probar el servicio de varias maneras. Una de las más conocidas es Postman, donde enviamos un GET, POST, PUT o DELETE y revisamos qué responde la API. Así podemos detectar rápido si el problema está en la petición o en Angular.”

Persona 3

“¿Y si quiero ver qué está pasando directamente en el navegador?”

Persona 2

“Podemos abrir las herramientas de desarrollador y revisar la pestaña Network. Ahí vemos la petición, la URL, el método, el código de respuesta y los datos que regresan.”

Persona 4

“O sea, si hago clic en “ver productos” y no aparece nada, no tendría que adivinar qué pasó.”

Persona 2

“Exacto. Revisamos Network y podemos encontrar pistas. Por ejemplo, un 200 indica que la petición respondió correctamente, mientras que un 404 nos haría revisar si la ruta existe.”

Persona 1

“Y para cerrar, quiero que lo juntes todo en una sola explicación. Como si estuvieras respondiendo la pregunta final de la entrevista.”

Persona 2

“Claro. El Service organiza la lógica para trabajar con los datos. HttpClient permite que ese Service se comunique mediante peticiones HTTP. JSON Server nos da una API simulada para practicar y herramientas como Postman y Network nos ayudan a comprobar que las peticiones y respuestas estén funcionando bien.”

Persona 3

“Entonces cada parte tiene una función distinta y, juntas, forman un flujo bastante completo.”

Persona 4

Indicacion: Dilo con una sonrisa y como una broma breve.

“Y lo mejor es que, cuando algo falla, podemos encontrar en qué parte falló en lugar de culpar al pobre Angular.”

Persona 1

Indicacion: Sonríe y cierra como una entrevista real.

“Bueno, esa respuesta sí pasó la entrevista. Gracias por participar.”

APOYO RÁPIDO PARA RECORDAR LOS CONCEPTOS

Concepto	Qué deben transmitir
Services	Separan la lógica de acceso a datos del componente y permiten reutilizar esa lógica.
HttpClient	Realiza peticiones HTTP como GET, POST, PUT y DELETE para comunicarse con una API.
JSON Server	Permite levantar una API de práctica a partir de un archivo JSON, útil cuando aún no existe un backend.
Postman	Sirve para enviar peticiones y revisar respuestas de la API de forma independiente de la interfaz.
DevTools - Network	Permite observar las peticiones que hace el navegador y revisar rutas, métodos y respuestas.

Ejemplo central que pueden recordar

Angular llama a un Service → el Service usa HttpClient → hace un GET a /productos → JSON Server responde los datos → Angular los muestra → Postman y Network ayudan a comprobar que todo respondió bien.

Duración aproximada

Entre 3 y 4 minutos, hablando a un ritmo normal y dejando pequeñas pausas para que se sienta como una conversación.

Importante

No es necesario memorizar palabra por palabra. Solo recuerden el orden de la conversación y el ejemplo central. Cada persona mantiene siempre el mismo número durante toda la dinámica. Las indicaciones de actuación están colocadas antes de la oración que deben interpretar.